P0595R1, 2018-05-04
EWG, LEWG

Richard Smith (richard@metafoo.co.uk)
Andrew Sutton (andrew.n.sutton@gmail.com)
Daveed Vandevoorde (daveed@edg.com)

`std::is_constant_evaluated()`

Introduction and Motivation

We propose a "magical" library function that is predeclared as follows:

```
namespace std {
  constexpr bool is_constant_evaluated() noexcept;
}
```

(No standard header inclusion or module import is needed to make it available.)

This function could be used as follows:

```
constexpr double power(double b, int x) {
  if (std::is_constant_evaluated() && x >= 0) {
    // A constant-evaluation context: Use a
    // constexpr-friendly algorithm.
    double r = 1.0, p = b;
    unsigned u = (unsigned)x;
    while (u != 0) {
      if (u & 1) r *= p;
      u /= 2;
      p *= p;
    }
    return r;
  } else {
    // Let the code generator figure it out.
    return std::pow(b, (double)x);
  }
}

constexpr double kilo = power(10.0, 3);  // (1)
int n = 3;
double mucho = power(10.0, n);  // (2)
double thousand() {
  return power(10.0, 3);
}
```

Call (1) occurs in a constant-expression context, and, therefore, `std::in_constexpr_call()` will be `true` during the computation of `power(10.0, 3)`, which in turn allows the evaluation to complete as a constant-expression.

Call (2) isn't a constant-expression because `n` cannot be converted to an rvalue in a constant-expression context. So it will be evaluated in a context where `std::is_constant_evaluated()` is `false`; this is known at translation time, and the run-time code generated for the function can therefore easily be reduced to the equivalent of just

```
inline double power'(double b, int x) {
  return std::pow(b, (double)x);
}
```

Call (3) is a core constant expression, but an implementation is not required to evaluate it at compile time. We therefore specify that it causes `std::is_constant_evaluated()` to produce `false`. It's tempting to leave it unspecified whether `true` or `false` is produced in that case, but that raises significant semantic concerns: The answer could then become inconsistent across various stages of the compilation. For example:

```
int *p, *invalid;
constexpr bool is_valid() {
  return std::is_constant_evaluated() ? true : p != invalid;
}
constexpr int get() { return is_valid() ? *p : abort(); }
```

This example tries to count on the fact that constexpr evaluation detects undefined behavior to avoid the non-constexpr-friendly call to `abort()` at compile time. However, if `std::is_constant_evaluated()` can return `true`, we now end up with a situation where

an important run-time check is by-passed.

Details

Ideally, we'd like this function to return `true` when it is evaluated at compile time, and `false` otherwise. However, the standard doesn't actually make a distinction between "compile time" and "run time", and hence a more careful specification is needed, one that fits the standard framework of "constant expressions".

Our approach is to precisely identify a set of expressions that are "required to be constant-evaluated" (a new technical phrase) and specify that our new function returns `true` during the evaluation of such expressions and `false` otherwise.

Specifically, we include two kinds of expressions in our set of expressions "required to be constant-evaluated". The first kind is straightforward: Expressions in contexts where the standard already requires a constant result, such as the dimension of an array or the initializer of a constexpr variable.

The second kind is a little more subtle: Expressions appearing in the initializers of variables that are not constexpr, but whose "constant-ness" has a significant semantic effect. Consider the following example:

```
template<int> struct X {};
constexpr auto f() {
  int const N = std::is_constant_evaluated() ? 13 : 17;
  X<N> x1;
  X<std::is_constant_evaluated() ? 13 : 17> x2;
}
```

We want to ensure that `x1` and `x2` have the same type. However, the initializer of `N`, in itself, it not required to have a constant value. It's only when we use `N` as a template argument later on that the requirement of a constant value arises, but by then a compiler already has committed its decision regarding the result of `std::is_constant_evaluated()`. In contrast, the second evaluation of `std::is_constant_evaluated()` falls squarely in the first kind of expressions "required to be constant-evaluated".

Our approach for the second kind, then, is to specify that the initializer for a variable whose "constant-ness" matters for the semantics of the program is also "required to be constant-evaluated" if evaluating it with `std::is_constant_evaluated() == true` would produce a constant. (The variables for which "constant-ness" matters are those of reference type or non-volatile const integral type because they can be used to form constant values, as well as those of non-automatic storage duration because the "constant-ness" of their initializers can affect initialization timing.) This implies that compilers have to perform a "tentative constant evaluation" for the initializers of such variables. Fortunately, that is already what current implementations do.

It is worth noting that despite the precise specification proposed here, this feature has potential sharp edges. The following example illustrates that:

```
constexpr int f() {
  const int n = std::is_constant_evaluated() ? 13 : 17; // n == 13
  int m = std::is_constant_evaluated() ? 13 : 17; // m might be 13 or 17 (see below)
  char arr[n] = {}; // char[13]
  return m + sizeof(arr);
}
int p = f(); // m == 13; initialized to 26
int q = p + f(); // m == 17 for this call; initialized to 56
```

The initializer of `p` (which has static storage duration and therefore is "required to be constant-evaluated") does produce a constant value with `std::is_constant_evaluated() == true`, and thus that is the value used for the actual evaluation: This results in `m` being initialized to `13` during the call to `f()`. In contrast, The initializer of `q` (also "required to be constant-evaluated") does *not* produce a constant value with `std::is_constant_evaluated() == true` because the use of `p` in the initializer is not a core constant expression. Thus the tentative evaluation with `std::is_constant_evaluated() == true` is discarded and the actual evaluation finds `std::is_constant_evaluated() == false`: This time, `m` is initialized to `17` during the call to `f()`. In other words, identical-looking expressions produce distinct results. A reasonable coding guideline in this context is that any dependence on `std::is_constant_evaluated()` should not affect the result of the computation: The initialization of `m` in the above example is thus arguably poor practice.

Notes

As the introductory example shows, `std::is_constant_evaluated()` is useful to enable alternative implementations of functions for compile time when the corresponding implementation for run time would not comply to the constraints of core constant expressions. A forthcoming important special case of this principle is `std::string`: P0578 (already approved by EWG) enables constexpr destructors, allocation, and deallocation, which in principle allows for the support of constexpr container types. However, `std::string` implementations typically include a "short string optimization" that is unfriendly to the constexpr

evaluation constraints: With the facility presented here, the implementation of `std::string` can avoid the short string optimization when evaluation happens at compile time. (In turn, the ability to produce `std::string` objects at compile time is expected to be beneficial for reflection interfaces.)

A previous version of this paper was presented in Kona (2017) using the special-purpose notation `constexpr()` instead of a (magic) library function call. The following two poll results were recorded at the time:

> The constexpr operator as presented?
> SF: 4 | F: 13 | N: 7 | A: 2 | SA: 2
>
> Same feature with a magic library function?
> SF: 5 | F: 12 | N: 5 | A: 2 | SA: 1

Wording Changes

Add a new section at the end of clause 21 [language.support]

21.12 Constexpr Support [support.constexpr]

21.12.1 Constexpr evaluation context [support.constexpr.is_constant_evaluated]

1. The following function is predefined in every translation unit:

```
namespace std {
  constexpr bool is_constant_evaluated() noexcept {
    ...
  }
}
```

2. *Remarks:*An expression `e` is *required to be constant-evaluated* if:
   - it is a *constant-expression* (_expr.const_), or
   - it is the *condition* of a *constexpr if* statement (_stmt.if_), or
   - it is the initializer of a constexpr variable (_dcl.constexpr_), or
   - it is a *constraint-expression* (_temp.constr.decl_) (possibly one formed from the *constraint-logical-or-expression* of a *requires-clause*), or
   - it is the initializer of a variable of reference type or of non-volatile const-qualified integral or enumeration type or of non-automatic storage duration, and `e` would be a constant expression (_expr.const_) if it were required to be constant-evaluated.
3. *Returns:* `true` if evaluation of the call occurs within the evaluation of an expression that is required to be constant-evaluated, `false` otherwise.
4. Every invocation of `std::is_constant_evaluated()` is a core constant expression.
5. [*Example:*

```
template<int> struct X {}
X<std::is_constant_evaluated()> x; // type X<true>
int y;
int a = std::is_constant_evaluated() ? y : 1; // initializes a to 1
int b = std::is_constant_evaluated() ? 2 : y; // initializes b to 2
int c = y + std::is_constant_evaluated() ? 2 : y; // initializes c to 2*y

constexpr int f() {
  const int n = std::is_constant_evaluated() ? 13 : 17; // n == 13
  int m = std::is_constant_evaluated() ? 13 : 17; // m might be 13 or 17 (see below)
  char arr[n] = {}; // char[13]
  return m + sizeof(arr);
}
int p = f(); // m == 13; initialized to 26
int q = p + f(); // m == 17 for this call; initialized to 56
```

— end example]